NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

DATABASE MANAGEMENT SYSTEM REPORT

Personal Finance Management System

Course: Database Management Systems
Supervisor: Tran Hung
Student: Ngo Tuan Bach
Student ID: 11245846
Class: DSEB 66B
Academic Year: 2025–2026

PROJECT 13: PERSONAL FINANCE MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to develop a system that enables users to effectively manage personal income and expenses, track financial activities, analyze spending habits, and optimize savings through data-driven reports and alerts.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage personal financial data including income, expenses, bank accounts, and spending categories.
- Allow users to monitor their financial health through detailed summaries and visual reports.

Main Functionalities

- User profile management.
- Income and expense entry, update, and categorization.
- Bank account tracking and balance history.
- Daily, monthly, and yearly financial summaries.
- Budget planning and spending limit alerts.
- Reporting on trends and category-wise expenditures.

Database Design and Implementation

1. **Data Model Design**
   - Create an ER diagram modeling users, income, expenses, and accounts.
   - Convert to relational schema with PKs, FKs, and necessary constraints.
2. **Table Structures**
   - Users (UserID, UserName, Email, PhoneNumber)
   - Income (IncomeID, UserID, Amount, IncomeDate, Description)
   - ExpenseCategories (CategoryID, CategoryName)
   - Expenses (ExpenseID, UserID, CategoryID, Amount, ExpenseDate, Description)
   - BankAccounts (AccountID, UserID, BankName, Balance)
3. **Sample Data**
   - Populate each table with 5–10 representative records.
   - Create a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Enhance performance for user queries and expense lookups.
- **Views:** Create monthly income/expense summaries, and category-wise spending views.
- **Stored Procedures:** Automate balance updates and monthly closure operations.
- **User Defined Functions:** Compute total income, total expenses, or budget status.
- **Triggers:** Automatically update bank account balance upon income or expense entry.

Database Security and Administration

- Secure user information with authentication roles.
- Implement access controls to protect personal financial data.
- Ensure backup and recovery procedures are in place.
- Optimize query performance for report generation.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Finance Tracker:** Build scripts for users to input transactions, track expenses, and view summaries.
- **Reporting:** Generate graphical and tabular financial reports.
- **User Interface:** Optional CLI or GUI to enhance user experience.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize key achievements of the system.
- Recommend features such as saving goals, predictive analytics, or integration with mobile banking.

References

- Include all resources, documentation, and libraries used during the project.

Contents

1 Introduction **7**

1.1 Background . . . 7

1.2 Problem Statement . . . 7

1.3 Project Objectives . . . 7

1.4 Project Scope . . . 8

1.4.1 Core Scope . . . 8

1.4.2 Extended Scope . . . 8

1.5 Technology Stack . . . 8

2 System Requirement Analysis **9**

2.1 Functional Requirements . . . 9

2.1.1 User and Authentication Management . . . 9

2.1.2 Income Management . . . 9

2.1.3 Expense Management . . . 9

2.1.4 Bank Account Tracking . . . 9

2.1.5 Budget Management . . . 9

2.1.6 Saving Goal Management . . . 9

2.1.7 Reporting and Visualization . . . 9

2.1.8 Transaction Import . . . 9

2.2 Non-functional Requirements . . . 10

3 System Architecture **10**

3.1 Overall Architecture . . . 10

3.2 Repository Structure . . . 10

3.3 Frontend Architecture . . . 10

3.4 Backend Architecture . . . 11

3.5 Database Architecture . . . 11

4 Database Design **11**

4.1 Database Overview . . . 11

4.2 Core ERD . . . 11

4.2.1 Core Entities . . . 12

4.2.2 Core Relationships . . . 12

4.2.3 Design Rationale . . . 13

4.3 Authentication and Security Supporting ERD . . . 14

4.3.1 Authentication and Security Entities . . . 14

4.3.2 Authentication and Security Relationships . . . 15

4.3.3 Design Rationale . . . 15

4.4 Transaction Import and Linking Supporting ERD . . . 16

4.4.1 Transaction Import and Linking Entities . . . 16
4.4.2 Transaction Import and Linking Relationships . . . 17
4.4.3 Design Rationale . . . 18
4.5 Full ERD . . . 19
4.6 Normalization . . . 19
4.6.1 First Normal Form (1NF) . . . 19
4.6.2 Second Normal Form (2NF) . . . 19
4.6.3 Third Normal Form (3NF) . . . 19
4.7 Constraints and Data Integrity . . . 19
4.8 Indexing Strategy . . . 20

5 SQL Implementation **20**
5.1 Schema Implementation . . . 20
5.2 Sample Data . . . 21
5.3 Views . . . 21
5.4 Stored Procedures . . . 23
5.5 User-Defined Functions . . . 23
5.6 Triggers . . . 24
5.7 Security SQL . . . 25
5.8 Migrations . . . 26

6 Backend Implementation **26**
6.1 Backend Overview . . . 26
6.2 Backend Architecture Flow . . . 26
6.3 Main Backend Components . . . 26
6.4 Backend Responsibilities . . . 27

7 Frontend Implementation **27**
7.1 Frontend Overview . . . 27
7.2 Frontend Interaction Flow . . . 27
7.3 Main Frontend Features . . . 27
7.4 User Interface Design . . . 28

8 Testing and Evaluation **28**
8.1 Testing Objectives . . . 28
8.2 Database Testing . . . 28
8.3 Backend Testing . . . 29
8.4 Frontend Testing . . . 30
8.5 Deployment Testing . . . 30
8.6 Test Case Summary . . . 31
8.7 Evaluation . . . 32

9 Discussion **32**
9.1 Security and Data Integrity Summary 32
9.2 Achievements 33
9.3 Limitations 33
9.4 Future Improvements 33

10 Conclusion **34**

References **35**

Appendix **36**

A Source Code Structure **36**

B SQL Script Inventory **36**

C Full ERD **37**

D Repository and Demo Links **37**

Abstract

This report presents the design and implementation of the Personal Finance Management System, a full-stack database-driven web application for managing personal financial activities. The system supports user management, authentication, bank accounts, income and expense tracking, expense categorization, monthly budget planning, fixed expenses, saving goals, goal contributions, transaction import, financial reports, and administrative functions. The database is implemented in MySQL under the schema name Personal_Finance, while the backend is developed with FastAPI and Python, and the frontend is implemented with Next.js, TypeScript, and Tailwind CSS. The report describes the system requirements, architecture, database design, ERD diagrams, SQL implementation, backend and frontend modules, testing process, security mechanisms, limitations, and future improvements. The project demonstrates how relational database concepts such as primary keys, foreign keys, normalization, constraints, indexes, views, stored procedures, functions, and triggers can be applied to a practical personal finance management system.

1 Introduction

1.1 Background

Personal finance management is an important activity for individuals who want to monitor their income, control expenses, plan monthly budgets, and achieve long-term saving goals. Without a structured system, users may find it difficult to understand where their money goes, whether they are overspending, and how much they can save each month.

The Personal Finance Management System was developed to solve this problem by providing a centralized platform for recording and analyzing personal financial data. The project focuses on database design and implementation, while also delivering a complete web application with backend APIs and a frontend user interface.

1.2 Problem Statement

Many users manage income and expenses manually using notes, spreadsheets, or banking applications that do not provide full financial planning functions. These methods may lack category-based spending analysis, budget warnings, saving goal tracking, transaction import, and historical reports.

This project addresses the following problems:

- Users need to store personal financial data in a structured and secure database.
- Users need to track income and expenses by account, category, and date.
- Users need monthly budget planning and spending alerts.
- Users need to monitor saving goals and goal contributions.
- Users need reports and visualizations to understand financial behavior.
- Administrators need basic user management capabilities.

1.3 Project Objectives

The main objective of the project is to design and implement a relational database system for personal finance management. The specific objectives are:

- Design a normalized MySQL database schema named Personal_Finance.
- Implement core financial entities such as users, bank accounts, income, expenses, categories, budgets, and saving goals.

- Implement authentication-related tables for credentials, sessions, OTP codes, and audit logs.
- Implement transaction import tables for CSV/Excel import workflow.
- Use views, stored procedures, functions, triggers, constraints, and indexes to improve data integrity and reporting.
- Build a FastAPI backend to expose database operations through REST APIs.
- Build a Next.js frontend for user interaction and financial visualization.
- Deploy the system using Docker Compose locally and Railway globally.

1.4 Project Scope

The project scope includes both core database requirements and extended application features.

1.4.1 Core Scope

The core scope includes:

- User profile management.
- Bank account management.
- Income and expense tracking.
- Expense category management.
- Monthly budget settings and budget plans.
- Saving goals and goal contributions.
- Financial reports and account balance history.

1.4.2 Extended Scope

The extended scope includes:

- Authentication, session tokens, OTP, and audit logs.
- Admin user management.
- CSV/Excel transaction import.
- Smart budget guardrails and spending alerts.
- Vietnamese user interface.
- Railway deployment.

1.5 Technology Stack

Table 1: Technology stack used in the project

Layer	Technology	Purpose
Database	MySQL	Store relational financial data.
Backend	Python, FastAPI	Provide REST APIs and business logic.
Frontend	Next.js, TypeScript, Tailwind CSS	Build the user interface.
Database Driver	mysql-connector-python	Connect the backend application to MySQL.
Charts	Recharts	Display financial reports and visualizations.
Local Development	Docker Compose	Run the database, backend, and frontend locally.

Continued on next page

Table 1: Technology stack used in the project (continued)

Layer	Technology	Purpose
Deployment	Railway	Host MySQL, backend, and frontend services.
Database Tool	MySQL Workbench / MySQL Query Editor	Run SQL commands and inspect schema, data, and query results.
ERD Tool	ERDPlus	Design ERD diagrams.

2 System Requirement Analysis

2.1 Functional Requirements

2.1.1 User and Authentication Management

The system must support user registration, login, logout, session management, password reset, OTP-based account unlock, and administrator-level user management. This requirement ensures that financial data is associated with the correct user and protected by authentication.

2.1.2 Income Management

The system must allow users to create, update, delete, and view income records. Each income transaction is linked to a user and an internally managed bank account.

2.1.3 Expense Management

The system must allow users to create, update, delete, and view expense records. Each expense transaction is linked to a user, a bank account, and an expense category, allowing the system to analyze spending behavior by category.

2.1.4 Bank Account Tracking

The system must allow users to manage self-declared bank accounts inside the application. These accounts are not connected to real banking APIs; instead, their balances are maintained internally based on income and expense transactions entered by the user.

2.1.5 Budget Management

The system must support monthly budget settings, fixed expense setup, category-based budget plans, planned versus actual comparison, and warning or exceeded statuses when spending approaches or exceeds the planned budget.

2.1.6 Saving Goal Management

The system must allow users to create saving or debt repayment goals, assign goal categories, record contributions, and monitor goal progress over time.

2.1.7 Reporting and Visualization

The system must provide financial reports and charts, including income versus expense summaries, category-wise spending, budget comparison, saving goal progress, account balance history, and cash flow visualization.

2.1.8 Transaction Import

The system must support importing transactions from CSV or Excel files. The import process includes row preview, value parsing, duplicate detection, category suggestion, and import confirmation.

2.2 Non-functional Requirements

- **Data integrity:** The system must use primary keys, foreign keys, unique constraints, check constraints, and triggers to keep financial data consistent.
- **Security:** Passwords, session tokens, and OTP codes must not be stored in plain text.
- **Performance:** Indexes should support common lookups, dashboard queries, and financial reports.
- **Maintainability:** The project should separate database scripts, backend APIs, service logic, repository logic, and frontend components.
- **Usability:** The user interface should be clear, localized in Vietnamese, and suitable for everyday personal finance tracking.
- **Deployability:** The system should run locally using Docker Compose and be deployable globally using Railway.

3 System Architecture

3.1 Overall Architecture

The system follows a layered architecture. The user interacts with the Next.js frontend through a web browser. The frontend sends HTTP requests to the FastAPI backend through REST APIs. The backend uses a service layer to process business logic and repository classes to interact with the MySQL database.

3.2 Repository Structure

The project repository is organized into separate folders for database scripts, backend code, frontend code, shared Python application logic, and operational scripts.

Listing 1: Main repository structure

```
database/
  schema.sql
  sample_data.sql
  views.sql
  procedures.sql
  functions.sql
  triggers.sql
  security.sql
  migrations/

backend/
frontend/
  src/
    app/
    components/
    lib/
    providers/
    types/
python_app/
ops/
docs/
docker-compose.yml
README.md
```

3.3 Frontend Architecture

The frontend is implemented with Next.js and TypeScript. It provides pages for login, signup, dashboard, transactions, budgets, goals, reports, balance history, profile, and user management. Tailwind CSS is used for styling, while Recharts is used for financial charts.

3.4 Backend Architecture

The backend is implemented with FastAPI. It exposes REST APIs under the prefix `/api/v1`. The backend includes routers for authentication, dashboard, metadata, transactions, imports, reports, budgets, alerts, goals, and user management.

3.5 Database Architecture

The database layer is implemented in MySQL. It includes normalized tables, constraints, indexes, views, stored procedures, user-defined functions, and triggers. The database name is fixed as Personal_Finance.

4 Database Design

4.1 Database Overview

The system uses a MySQL relational database named `Personal_Finance`. The database stores the main financial data of the application, including users, authentication records, bank accounts, income, expenses, budgets, saving goals, transaction imports, sessions, OTP codes, and audit logs. The schema is designed with primary keys, foreign keys, constraints, indexes, views, triggers, functions, and stored procedures to ensure data consistency and support financial reporting.

Table 2: Database overview

Item	**Description**
Database name	`Personal_Finance`
DBMS	MySQL
Character set	`utf8mb4`
Collation	`utf8mb4_0900_ai_ci`
Main SQL file	`database/schema.sql`
Sample data file	`database/sample_data.sql`
Views file	`database/views.sql`
Stored procedures file	`database/procedures.sql`
Functions file	`database/functions.sql`
Triggers file	`database/triggers.sql`
Security setup file	`database/security.sql`

4.2 Core ERD

The Core ERD represents the main business entities of the system. It focuses on the tables that directly support the financial tracking workflow, including user profiles, bank accounts, income records, expense records, budget planning, and saving goals. These tables form the foundation of the system because most application features depend on them.

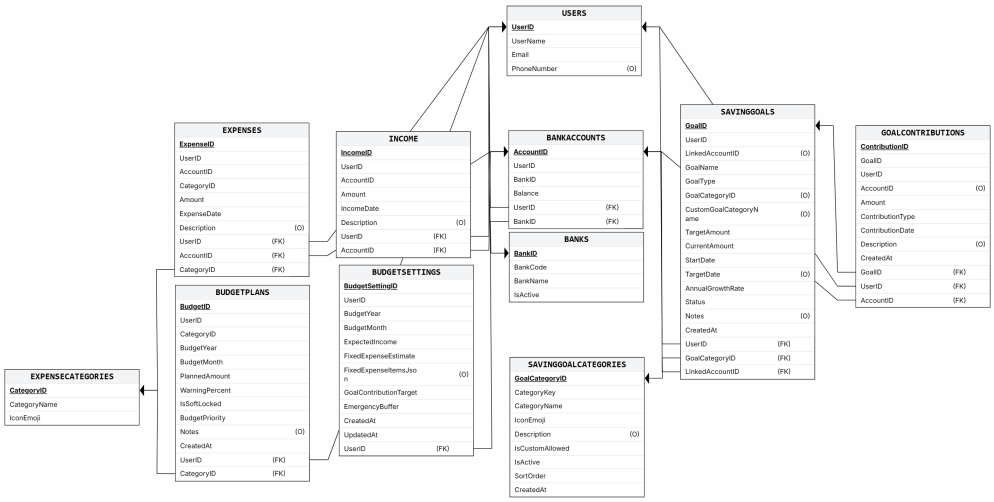

Figure 1: Core ERD of the Personal_Finance database

The core database design is organized around the `Users` table. Each user can own multiple bank accounts, record income and expense transactions, create monthly budgets, and define saving or debt repayment goals. This user-centered design ensures that all financial records are clearly separated by account owner.

4.2.1 Core Entities

Table 3: Core entities in the database

Entity	Purpose
`Users`	Stores user profile information.
`Banks`	Stores the predefined bank catalog, normalized separately from user accounts.
`BankAccounts`	Stores user-created bank accounts and their current balances.
`Income`	Stores income transactions.
`Expenses`	Stores expense transactions.
`ExpenseCategories`	Stores expense categories.
`BudgetSettings`	Stores monthly budget settings per user.
`BudgetPlans`	Stores category-based budget plans per user per month.
`SavingGoalCategories`	Stores predefined saving goal categories and indicates whether custom goal category names are allowed.
`SavingGoals`	Stores saving and debt repayment goals.
`GoalContributions`	Stores individual contributions toward saving goals.

4.2.2 Core Relationships

The relationships in the Core ERD are designed to preserve ownership and financial consistency. The `Users` table has one-to-many relationships with most financial tables because each user can manage multiple accounts, transactions, budgets, and goals. The `BankAccounts` table acts as a central link between users and transactions because income and expenses must be

assigned to a specific account. Expense transactions are also classified by `ExpenseCategories`, which allows the system to generate category-wise spending reports.

Table 4: Main relationships in the Core ERD

Relationship	Cardinality	Description
`Users - BankAccounts`	One-to-many	One user can create multiple internal bank accounts.
`Banks - BankAccounts`	One-to-many	One bank entry in the catalog can be referenced by multiple user accounts.
`Users - Income`	One-to-many	One user can record multiple income transactions.
`Users - Expenses`	One-to-many	One user can record multiple expense transactions.
`BankAccounts - Income`	One-to-many	Each income transaction is credited to one bank account.
`BankAccounts - Expenses`	One-to-many	Each expense transaction is deducted from one bank account.
`ExpenseCategories - Expenses`	One-to-many	One category can classify many expense transactions.
`Users - BudgetSettings`	One-to-many	One user can configure monthly budget settings across multiple months.
`Users - BudgetPlans`	One-to-many	One user can define multiple category-level budget plans.
`ExpenseCategories - BudgetPlans`	One-to-many	Each budget plan is assigned to one expense category.
`Users - SavingGoals`	One-to-many	One user can create multiple saving or debt repayment goals.
`Users - GoalContributions`	One-to-many	One user can create multiple contribution records toward saving goals.
`SavingGoals - GoalContributions`	One-to-many	One goal can receive multiple contribution records over time.
`BankAccounts - GoalContributions`	Optional one-to-many	A goal contribution can optionally be associated with one bank account.

4.2.3 Design Rationale

The Core ERD separates the main financial data into clear business entities to reduce redundancy and improve data consistency. The `Income` and `Expenses` tables are stored separately because they represent opposite types of transactions. Income records increase account balances, while expense records decrease them. This separation also makes transaction processing and financial reporting easier to understand.

The `ExpenseCategories` table is separated from `Expenses` to avoid storing repeated category names in every expense record. This design supports category-based analysis, such as identifying which types of expenses take the largest portion of a user's spending. Similarly, the budget module is divided into `BudgetSettings` and `BudgetPlans`. `BudgetSettings` stores the monthly budget configuration for each user, while `BudgetPlans` stores planned amounts for specific expense categories.

Regarding bank account modeling, the project requirement specification defined `BankAccounts` as a single table containing a `BankName` attribute. In this implementation, however, bank information is normalized into a separate `Banks` table, with `BankAccounts` referencing `Banks` via a `BankID` foreign key. This design decision eliminates the repetition of bank names across multiple user accounts, satisfies the Third Normal Form (3NF), and allows the application to maintain a controlled catalog of supported banks while still associating each user account with the correct institution.

The saving goal feature is modeled separately from daily transactions because saving goals represent long-term financial objectives. The `SavingGoals` table stores the goal information, while `GoalContributions` stores each contribution made toward a goal. This allows the system to calculate progress and monitor whether users are moving toward their financial targets.

Overall, the Core ERD provides a normalized and maintainable structure for the main financial features of the application. It supports income and expense tracking, internal bank account balance management, monthly budgeting, saving goal monitoring, and financial reporting.

4.3 Authentication and Security Supporting ERD

The Authentication and Security Supporting ERD describes the database structure used to support application-level authentication, account protection, OTP verification, audit logging, and session management. Although these tables are not directly related to daily financial transactions, they are important because they protect user accounts and ensure that financial data can only be accessed by authorized users.

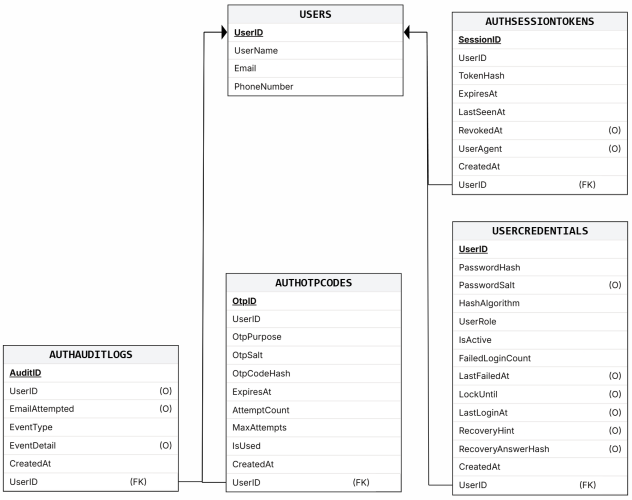

Figure 2: Authentication and Security Supporting ERD

The authentication design is centered around the `Users` table. The `Users` table stores basic profile information, while security-sensitive information is separated into supporting tables. This separation helps keep user profile data independent from login credentials, session tokens, OTP codes, and audit logs.

4.3.1 Authentication and Security Entities

Table 5: Authentication and security supporting entities

Entity	Purpose
`Users`	Stores basic user profile information.

Continued on next page

Table 5: Authentication and security supporting entities (continued)

Entity	Purpose
`UserCredentials`	Stores login credentials, roles, account status, and recovery information.
`AuthOtpCodes`	Stores hashed OTP codes for password reset and account unlock workflows.
`AuthAuditLogs`	Stores authentication-related audit events.
`AuthSessionTokens`	Stores hashed session tokens for authenticated user sessions.

4.3.2 Authentication and Security Relationships

Most supporting security records are linked to `Users` through the `UserID` foreign key. However, `AuthAuditLogs.UserID` may be nullable when recording failed login attempts or authentication events for unknown or unmatched email addresses. The relationship between `Users` and `UserCredentials` is one-to-one because each user should have one credential record. Other tables such as `AuthOtpCodes` and `AuthSessionTokens` have one-to-many relationships with `Users`, because one user can request multiple OTP codes and own multiple session tokens over time. `AuthAuditLogs` can also store many records for one user when the user is known, while still allowing unauthenticated or unmatched attempts to be logged.

Table 6: Main relationships in the Authentication and Security ERD

Relationship	Cardinality	Description
`Users - UserCredentials`	One-to-one	Each user has exactly one credential record storing password hash, role, and account status.
`Users - AuthOtpCodes`	One-to-many	One user can request multiple OTP codes for password reset or account unlock operations.
`Users - AuthAuditLogs`	Optional one-to-many	One known user can generate multiple authentication audit log records, while some audit logs may store only the attempted email.
`Users - AuthSessionTokens`	One-to-many	One user can hold multiple session tokens, for example when authenticated across different devices or browsers.

4.3.3 Design Rationale

The authentication and security tables are separated from the core financial tables to improve security, maintainability, and clarity. The `Users` table stores general profile information, while `UserCredentials` stores sensitive login-related data such as password hashes, user roles, account status, and recovery information.

The system avoids storing sensitive authentication data in plain text. Passwords, OTP codes, and session tokens are stored as hashes or salted hashes to reduce security risks if the database is accessed improperly.

The `AuthOtpCodes` table supports temporary verification workflows such as password reset and account unlocking. The `AuthSessionTokens` table manages authenticated sessions by tracking token expiration, last activity, and revocation status.

The `AuthAuditLogs` table records important authentication events, such as failed login attempts, account lock actions, password reset requests, and OTP-related activities.

Overall, this supporting ERD strengthens the authentication layer and provides traceability for account-related events.

4.4 Transaction Import and Linking Supporting ERD

The Transaction Import and Linking Supporting ERD describes the database structure used to support CSV/Excel transaction import, automatic category suggestion, duplicate detection, import confirmation, and the linking between financial transactions and saving goal contributions. This ERD extends the core financial model by adding supporting tables that help users import external transaction data and connect income or expense transactions with long-term financial goals.

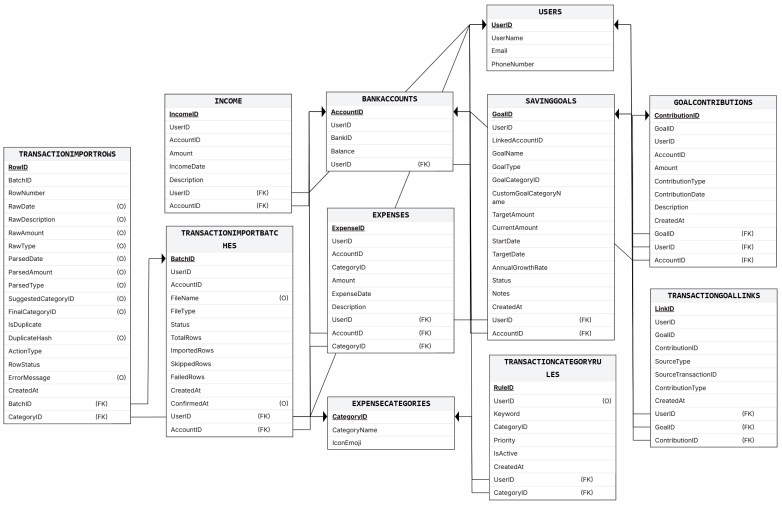

Figure 3: Transaction Import and Linking Supporting ERD

This supporting ERD is built around the transaction workflow. The import process starts with a file uploaded by the user. The file is stored as an import batch in `TransactionImportBatches`, while each row of the file is stored in `TransactionImportRows`. The system can parse raw transaction data, suggest categories based on predefined rules, detect duplicate transactions, and allow the user to confirm the final import action. In addition, the `TransactionGoalLinks` table records the relationship between imported or manually entered transactions and goal contributions.

4.4.1 Transaction Import and Linking Entities

Table 7: Transaction import and linking supporting entities

Entity	Purpose
`TransactionImportBatches`	Stores file-level information for each CSV or Excel import.
`TransactionImportRows`	Stores row-level transaction data during the import process.
`TransactionCategoryRules`	Stores keyword rules used to suggest expense categories.

Continued on next page

Table 7: Transaction import and linking supporting entities (continued)

Entity	Purpose
`TransactionGoalLinks`	Links income or expense transactions with goal contributions, enabling traceability between daily financial activities and long-term financial goals.
`Income`	Stores income transactions.
`Expenses`	Stores expense transactions.
`SavingGoals`	Stores saving or debt repayment goals.
`GoalContributions`	Stores contribution records toward saving goals.
`Users`	Stores the owner of transactions, imports, rules, and goals.
`BankAccounts`	Stores the bank account associated with transactions and imports.
`ExpenseCategories`	Stores categories used to classify expense transactions.

4.4.2 Transaction Import and Linking Relationships

The transaction import workflow is user-centered. Each import batch belongs to one user and one bank account. Each batch contains multiple import rows, and each import row can be parsed, categorized, marked as duplicate, skipped, failed, or confirmed for import. Category suggestion is supported by `TransactionCategoryRules`, which map user-defined keywords to `ExpenseCategories`.

The linking workflow connects normal financial transactions with saving goal contributions. A record in `TransactionGoalLinks` connects one source transaction to one goal contribution. The source transaction may come from either `Income` or `Expenses`, depending on the value of `SourceType`.

Table 8: Main relationships in the Transaction Import and Linking ERD

Relationship	Cardinality	Description
`Users - TransactionImportBatches`	One-to-many	One user can create multiple transaction import batches.
`BankAccounts - TransactionImportBatches`	One-to-many	One bank account can be associated with multiple imported transaction files.
`TransactionImportBatches - TransactionImportRows`	One-to-many	One import batch contains multiple imported rows from the uploaded file.
`ExpenseCategories - TransactionImportRows`	One-to-many	One expense category can be used as the suggested or final category for many imported rows.
`Users - TransactionCategoryRules`	One-to-many	One user can define multiple keyword-based category rules.
`ExpenseCategories - TransactionCategoryRules`	One-to-many	One expense category can be referenced by multiple category suggestion rules.
`Users - TransactionGoalLinks`	One-to-many	One user can have multiple links between transactions and goal contributions.

Continued on next page

Table 8: Main relationships in the Transaction Import and Linking ERD (continued)

Relationship	Cardinality	Description
`SavingGoals - TransactionGoalLinks`	One-to-many	One saving goal can be linked to multiple transaction-related contributions.
`GoalContributions - TransactionGoalLinks`	Optional one-to-one	Each transaction-goal link refers to exactly one goal contribution, while a manual contribution may exist without a transaction-goal link.
`Income - TransactionGoalLinks`	Conditional	If `SourceType = INCOME`, then `SourceTransactionID` refers to `Income.IncomeID`.
`Expenses - TransactionGoalLinks`	Conditional	If `SourceType = EXPENSE`, then `SourceTransactionID` refers to `Expenses.ExpenseID`.

Special Notes

- **Polymorphic reference in `TransactionGoalLinks`:** `SourceTransactionID` can refer to either `Income.IncomeID` or `Expenses.ExpenseID`, depending on the value of `SourceType`. Because MySQL does not natively support polymorphic foreign keys, this relationship must be validated at the backend application layer.
- **Optional category references in `TransactionImportRows`:** `SuggestedCategoryID` stores the category proposed by the system via keyword matching, while `FinalCategoryID` stores the category confirmed by the user before import. Both fields reference `ExpenseCategories.CategoryID` and are nullable to allow rows where no category has yet been assigned.
- **Duplicate detection:** `IsDuplicate` and `DuplicateHash` are used to detect repeated imported transactions and prevent the creation of duplicate financial records.

4.4.3 Design Rationale

The transaction import design separates file-level data from row-level data to make the import process easier to control. A batch represents one uploaded file, while each row can be reviewed, corrected, skipped, or confirmed independently. This structure allows the system to track both the overall import status and the detailed outcome of each imported transaction row.

The category rule mechanism is included to reduce manual effort during transaction import. By matching transaction descriptions against user-defined keywords stored in `TransactionCategoryRules`, the system can propose suitable expense categories before the user confirms the final classification.

The transaction-goal linking design improves traceability between daily financial activities and long-term saving objectives. Rather than embedding goal-related logic directly within the `Income` or `Expenses` tables, the system stores these associations in the separate `TransactionGoalLinks` table. This approach preserves the integrity of core transaction records while still allowing the system to identify which financial transaction contributed to a specific saving or debt repayment goal.

Overall, this supporting ERD improves the import workflow, reduces duplicate transaction records, supports automatic category suggestion, and maintains a clear and auditable connection between transactions and saving goals.

4.5 Full ERD

The Full ERD contains all nineteen tables in the `Personal_Finance` database. Because it is larger and more detailed than the individual supporting diagrams, it is placed in Appendix C for reference.

4.6 Normalization

The database schema follows normalization principles to reduce redundancy, avoid update anomalies, and improve data consistency. The design is organized so that each table stores one main type of data and relationships are represented through primary keys and foreign keys.

4.6.1 First Normal Form (1NF)

The schema satisfies First Normal Form because table attributes store atomic values and each record is uniquely identified by a primary key. For example, `Users` stores one user per row, `Income` stores one income transaction per row, and `Expenses` stores one expense transaction per row. Repeating groups are avoided by separating related data into different tables instead of storing multiple values in one column.

4.6.2 Second Normal Form (2NF)

The schema satisfies Second Normal Form because non-key attributes depend on the whole primary key of their table. Most tables use a single surrogate primary key, such as `UserID`, `IncomeID`, `ExpenseID`, or `GoalID`, which reduces the risk of partial dependency. Relationship data is stored through foreign keys. For example, `Income` and `Expenses` reference `Users` and `BankAccounts` instead of repeating user or account details inside transaction records.

4.6.3 Third Normal Form (3NF)

The schema satisfies Third Normal Form because non-key attributes do not depend on other non-key attributes. Lookup and supporting data are separated into independent tables. For example, bank catalog information is stored in `Banks`, while user-owned accounts are stored in `BankAccounts`. Expense category names are stored in `ExpenseCategories`, while expense records only reference the category through `CategoryID`. Similarly, `Users` stores general profile data, while `UserCredentials` stores authentication-related data.

Overall, the normalization design keeps the schema clean and maintainable. It reduces repeated values, improves consistency, and allows the system to manage financial transactions, budgets, goals, authentication, and reports more reliably.

4.7 Constraints and Data Integrity

The database uses several integrity mechanisms to ensure that financial data remains accurate and consistent.

- **Primary keys** uniquely identify records in each table. Most tables use a numeric auto-incremented key, such as `UserID`, `IncomeID`, or `ExpenseID`.
- **Foreign keys** enforce relationships between related tables. For example, expense records are linked to users, bank accounts, and expense categories.
- **Unique constraints** prevent duplicate important values, such as user emails, phone numbers, bank codes, bank names, and repeated monthly budget plans for the same user and category.
- **Check constraints** enforce valid financial values and period values, such as positive transaction amounts, non-negative balances, valid warning percentages, and valid budget months.
- **Triggers** keep `BankAccounts.Balance` synchronized when income or expense records are inserted, updated, or deleted. They also help prevent expense transactions when the selected account does not have enough balance.

Together, these mechanisms protect the database from invalid records and help maintain reliable financial data.

4.8 Indexing Strategy

Indexes are created to improve query performance for the most common access patterns in the application. Because most features are user-scoped and time-scoped, the indexing strategy prioritizes composite indexes on `UserID` combined with relevant date, period, status, or lookup columns.

- **Transaction reporting:** Indexes such as `Income(UserID,IncomeDate)` and `Expenses(UserID,ExpenseDate)` support dashboard and monthly summary queries. Additional indexes on `Income(IncomeDate)` and `Expenses(ExpenseDate,CategoryID)` support cross-user reporting and category-wise spending analysis.
- **Budget management:** Indexes on `BudgetPlans(UserID,BudgetYear,BudgetMonth)` and `BudgetSettings(UserID,BudgetYear,BudgetMonth)` support monthly budget lookups. An additional index on `BudgetPlans(UserID,BudgetYear,BudgetMonth,IsSoftLocked,BudgetPriority)` supports filtered retrieval of locked or high-priority budget lines.
- **Authentication and sessions:** Indexes on `AuthSessionTokens(UserID,ExpiresAt)` and `AuthSessionTokens(ExpiresAt,RevokedAt)` support session validation and expired-session cleanup. OTP-related indexes support OTP lookup, expiry checks, and audit trail retrieval.
- **Saving goals and contributions:** Indexes on `SavingGoals(UserID,Status)`, `SavingGoals(UserID,TargetDate)`, and `GoalContributions(GoalID,ContributionDate)` support goal progress queries and contribution history lookup.
- **Transaction import:** Indexes on `TransactionImportBatches(UserID,CreatedAt)`, `TransactionImportRows(BatchID,RowNumber)`, and `TransactionImportRows(DuplicateHash)` support batch retrieval, row-by-row access, and duplicate detection during file import.

Overall, the indexing strategy is aligned with the main query patterns of the system, including dashboard loading, financial reporting, budget analysis, session validation, goal tracking, and transaction import.

5 SQL Implementation

This section describes the SQL implementation of the `Personal_Finance` database. The implementation is divided into several SQL files, including `schema.sql`, `sample_data.sql`, `views.sql`, `procedures.sql`, `functions.sql`, `triggers.sql`, `security.sql`, and the `migrations/` folder. These files define the database structure, insert sample data, create reporting views, implement stored routines, automate account balance updates, and configure database-level access control.

The screenshots in this chapter are captured from MySQL query results. Each screenshot is shown together with the SQL command used to produce it.

5.1 Schema Implementation

The main schema is defined in `database/schema.sql`. This file creates the `Personal_Finance` database and defines the main relational structure of the system. It includes tables, primary keys, foreign keys, unique constraints, check constraints, and indexes. The schema covers user management, authentication, bank accounts, income, expenses, budgets, saving goals, transaction import, sessions, OTP verification, and audit logging.

Most tables use a numeric auto-incremented surrogate primary key. The main exception is `UserCredentials`, which uses `UserID` as both its primary key and foreign key to `Users`. Foreign key constraints are used to maintain relationships between related tables. In

addition, `BankAccounts(AccountID,UserID)` is used as a composite unique key to ensure that transactions are linked only to accounts owned by the correct user.

The following SQL command was used to verify the created tables and views:

```
USE Personal_Finance;

SHOW FULL TABLES;
```

Tables_in_personal_finance	Table_type
authauditlogs	BASE TABLE
authotpcodes	BASE TABLE
authsessiontokens	BASE TABLE
bankaccounts	BASE TABLE
banks	BASE TABLE
budgetplans	BASE TABLE
budgetsettings	BASE TABLE
expensecategories	BASE TABLE
expenses	BASE TABLE
goalcontributions	BASE TABLE
income	BASE TABLE
savinggoalcategories	BASE TABLE
savinggoals	BASE TABLE

(a) Base tables

transactioncategoryrules	BASE TABLE
transactionimportbatches	BASE TABLE
transactionimportrows	BASE TABLE
usercredentials	BASE TABLE
users	BASE TABLE
vw_account_balance_history	VIEW
vw_budget_vs_actual_mo...	VIEW
vw_category_wise_spending	VIEW
vw_monthly_income_expe...	VIEW
vw_saving_goal_progress	VIEW
vw_smart_budget_overview	VIEW
vw_spending_limit_alerts	VIEW
vw_total_expense_by_user	VIEW
vw_total_income_by_user	VIEW

(b) Remaining tables and views

Figure 4: MySQL result of `SHOW FULL TABLES` in the `Personal_Finance` database

Figure 4 shows that the database contains both base tables and reporting views after executing the schema and supporting SQL scripts.

The following SQL command was used to inspect the structure of the `BankAccounts` table:

```
DESCRIBE BankAccounts;
```

Field	Type	Null	Key	Default	Extra
AccountID	int	NO	PRI	NULL	auto_increment
UserID	int	NO	MUL	NULL	
BankID	int	NO	MUL	NULL	
Balance	decimal(15,2)	NO		0.00	

Figure 5: MySQL result of `DESCRIBE BankAccounts`

Figure 5 shows the structure of the `BankAccounts` table, including its primary key, foreign key columns, and balance field.

5.2 Sample Data

Sample data is inserted through `database/sample_data.sql`. This file provides representative test records for demonstration and validation. The sample data includes user profiles, banks, bank accounts, expense categories, income records, expense records, budget settings, budget plans, saving goal categories, saving goals, goal contributions, and transaction category rules.

Authentication credentials are initialized separately through the bootstrap authentication script instead of being hard-coded in the sample data file. This keeps the sample data useful for testing while avoiding fixed authentication secrets inside the SQL seed file.

5.3 Views

Reporting views are implemented in `database/views.sql`. Views simplify complex reporting queries by storing reusable `SELECT` logic in the database layer. This allows the backend

to retrieve dashboard and report data without rebuilding the same joins and aggregations repeatedly.

Table 9: Database views and their purposes

View	Purpose
`vw_total_income_by_user`	Summarizes total income for each user.
`vw_total_expense_by_user`	Summarizes total expenses for each user.
`vw_monthly_income_expense_net_saving`	Calculates monthly income, expense, and net saving.
`vw_category_wise_spending`	Summarizes spending by expense category.
`vw_budget_vs_actual_monthly`	Compares planned budget amounts with actual monthly spending.
`vw_spending_limit_alerts`	Returns budget categories that are in warning or exceeded status.
`vw_account_balance_history`	Computes chronological account running balances.
`vw_saving_goal_progress`	Calculates saving goal progress and remaining amount.
`vw_smart_budget_overview`	Provides a monthly overview of budget capacity and budget health.

The following SQL command was used to verify the budget comparison view:

```
SELECT *
FROM vw_budget_vs_actual_monthly
LIMIT 10;
```

BudgetID	UserID	UserName	CategoryID	CategoryName	BudgetYear	BudgetMonth	PlannedAmount	WarningPercent	SpentAmount	RemainingBudget	AlertLevel	AlertSortOrder
24	4	Pham Gia Huy	5	Mua s???m	2026	4	1400000.00	80.00	1500000.00	-100000.00	EXCEEDED	1
19	10	Ngo Tuan Bach	5	Mua s???m	2026	3	2500000.00	80.00	2800000.00	-300000.00	EXCEEDED	1
15	8	Phan Duc Long	3	Gi??o d???c	2026	3	2100000.00	80.00	2200000.00	-100000.00	EXCEEDED	1
11	6	Vu Khanh Nam	6	S???c kh???e	2026	3	1500000.00	80.00	1700000.00	-200000.00	EXCEEDED	1
5	3	Le Thu Ha	6	S???c kh???e	2026	3	1700000.00	80.00	1800000.00	-100000.00	EXCEEDED	1
3	2	Tran Quoc Bao	8	Thu?? nh??	2026	3	5800000.00	80.00	6000000.00	-200000.00	EXCEEDED	1
30	10	Ngo Tuan Bach	5	Mua s???m	2026	4	3200000.00	80.00	2800000.00	400000.00	WARNING	2
29	9	Dang Bao Chau	6	S???c kh???e	2026	4	1000000.00	80.00	900000.00	100000.00	WARNING	2
28	8	Phan Duc Long	3	Gi??o d???c	2026	4	2300000.00	80.00	2200000.00	100000.00	WARNING	2
26	6	Vu Khanh Nam	6	S???c kh???e	2026	4	600000.00	80.00	500000.00	100000.00	WARNING	2

Figure 6: MySQL result of `vw_budget_vs_actual_monthly`

Figure 6 shows the result of the budget comparison view. The view displays planned amounts, actual spending, remaining budgets, and alert levels for monthly budget analysis.

The following SQL command was used to verify the account balance history view:

```
SELECT *
FROM vw_account_balance_history
LIMIT 10;
```

UserID	UserName	AccountID	BankName	TransactionDate	TransactionType	SortOrder	ReferenceID	AmountSigned	RunningBalance
1	Nguyen Minh Anh	1	Vietcombank	2025-01-01 08:00:00	INCOME	1	16	20000000.00	20000000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-03 09:00:00	EXPENSE	2	16	-3500000.00	16500000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-10 19:00:00	EXPENSE	2	271	-2200000.00	14300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-18 08:30:00	EXPENSE	2	526	-1000000.00	13300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-01 08:00:00	INCOME	1	15	20000000.00	33300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-03 09:00:00	EXPENSE	2	15	-3500000.00	29800000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-10 19:00:00	EXPENSE	2	270	-2200000.00	27600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-18 08:30:00	EXPENSE	2	525	-1000000.00	26600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-03-01 08:00:00	INCOME	1	14	20000000.00	46600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-03-03 09:00:00	EXPENSE	2	14	-3500000.00	43100000.00

Figure 7: MySQL result of `vw_account_balance_history`

Figure 7 shows transaction-based running balances for a selected account. This view is used to analyze how income and expense transactions change account balance over time.

5.4 Stored Procedures

Stored procedures are implemented in `database/procedures.sql`. They centralize common write and summary operations, such as adding income, adding expenses, updating transactions, deleting transactions, managing budget plans, and generating monthly financial summaries.

Table 10: Stored procedures and their purposes

Procedure	Purpose
`sp_add_income`	Inserts a new income transaction.
`sp_add_expense`	Inserts a new expense transaction.
`sp_update_income`	Updates an existing income transaction.
`sp_update_expense`	Updates an existing expense transaction.
`sp_delete_income`	Deletes an income transaction.
`sp_delete_expense`	Deletes an expense transaction.
`sp_add_budget_plan`	Creates a category-based monthly budget plan.
`sp_update_budget_plan`	Updates an existing budget plan.
`sp_monthly_closure_summary`	Returns total income, total expense, and net saving for a selected month.

The following SQL command was used to test the monthly summary stored procedure:

```
CALL sp_monthly_closure_summary(2025, 4);
```

YearMonth	MonthlyIncome	MonthlyExpense	NetSaving
2025-04	156000000.00	85700000.00	70300000.00

Figure 8: MySQL result of executing `sp_monthly_closure_summary`

Figure 8 shows that the stored procedure returns monthly income, monthly expense, and net saving for the selected period.

5.5 User-Defined Functions

User-defined functions are implemented in `database/functions.sql`. These functions provide reusable financial calculations that can be used by reports, queries, procedures, or testing scripts.

Table 11: User-defined functions and their purposes

Function	Purpose
`fn_total_income_by_user`	Calculates total income for a user.
`fn_total_expense_by_user`	Calculates total expenses for a user.
`fn_net_saving_by_user`	Calculates net saving as total income minus total expense.
`fn_calculated_balance_by_account`	Calculates transaction-derived balance for a specific account.

The following SQL command was used to test the user-defined financial functions:

```sql
SELECT
    fn_total_income_by_user(1) AS TotalIncome,
    fn_total_expense_by_user(1) AS TotalExpense,
    fn_net_saving_by_user(1) AS NetSaving;
```

	TotalIncome	TotalExpense	NetSaving
▶	348000000.00	115200000.00	232800000.00

Figure 9: MySQL result of user-defined financial functions

Figure 9 shows that the financial functions correctly return total income, total expense, and net saving for a selected user.

5.6 Triggers

Triggers are implemented in `database/triggers.sql`. They automatically keep `BankAccounts.Balance` synchronized with income and expense transactions. The database defines triggers for `INSERT`, `UPDATE`, and `DELETE` operations on both `Income` and `Expenses`. It also includes validation triggers to prevent expense transactions when the selected account does not have enough balance.

Table 12: Triggers and their execution context

Trigger Name	Timing	Effect
`trg_expenses_before_insert_check_balance`	`BEFORE INSERT`	Rejects an expense if the selected account is invalid or has insufficient balance.
`trg_expenses_before_update_check_balance`	`BEFORE UPDATE`	Validates available balance before an expense update is applied.
`trg_income_after_insert_update_balance`	`AFTER INSERT`	Adds `NEW.Amount` to the related account balance.
`trg_income_after_update_update_balance`	`AFTER UPDATE`	Applies the income difference `NEW.Amount - OLD.Amount`.
`trg_income_after_delete_update_balance`	`AFTER DELETE`	Subtracts `OLD.Amount` from the related account balance.
`trg_expenses_after_insert_update_balance`	`AFTER INSERT`	Subtracts `NEW.Amount` from the related account balance.
`trg_expenses_after_update_update_balance`	`AFTER UPDATE`	Adjusts the balance according to the updated expense amount.
`trg_expenses_after_delete_update_balance`	`AFTER DELETE`	Adds `OLD.Amount` back to the related account balance.

The following SQL commands were used to verify the income insert trigger. First, the account balance was checked. Then, a test income transaction was inserted. Finally, the account balance was checked again.

```sql
SELECT AccountID, UserID, Balance AS BalanceBefore
FROM BankAccounts
WHERE UserID = 1 AND AccountID = 1;

INSERT INTO Income (UserID, AccountID, Amount, IncomeDate, Description)
VALUES (1, 1, 123456.00, NOW(), 'Trigger test income');

SELECT AccountID, UserID, Balance AS BalanceAfter
FROM BankAccounts
WHERE UserID = 1 AND AccountID = 1;
```

AccountID	UserID	BalanceBefore
1	1	212800000.00
NULL	NULL	NULL

(a) Balance before insert

15 09:08:53 INSERT INTO Income (UserID, AccountID, Amount, IncomeDate, Description) VALUES (1, 1, 1... 1 row(s) affected

(b) Income inserted

AccountID	UserID	BalanceAfter
1	1	212923456.00
NULL	NULL	NULL

(c) Balance after insert

Figure 10: MySQL verification of account balance update after trigger execution

Figure 10 demonstrates that after inserting a new income record, the account balance is automatically increased by the trigger. This verifies that the database trigger logic keeps account balances synchronized with transaction records.

After the screenshot was captured, the test transaction was removed using the following SQL commands to restore the database state:

```sql
SELECT IncomeID, UserID, AccountID, Amount, Description
FROM Income
WHERE Description = 'Trigger test income'
ORDER BY IncomeID DESC
LIMIT 1;

DELETE FROM Income
WHERE Description = 'Trigger test income'
ORDER BY IncomeID DESC
LIMIT 1;
```

5.7 Security SQL

Database-level access control is implemented in `database/security.sql`. This file creates MySQL users that are separate from application-level users stored in `Users` and `UserCredentials`. This separation allows the system to apply the principle of least privilege at the database engine level.

The `pf_app` user is intended for backend runtime operations. It is granted `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `SHOW VIEW`, and `EXECUTE` privileges on the `Personal_Finance` database. These privileges allow the backend to perform normal application operations while preventing structural operations such as `CREATE TABLE`, `DROP`, or `ALTER`.

The following SQL command was used to verify the privileges of the `pf_app` user:

```sql
SHOW GRANTS FOR 'pf_app'@'%';
```

Grants for pf_app@%:

GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE, SHOW VIEW ON `personal_finance`.* TO `pf_app`@`%`

Figure 11: MySQL result of database privileges assigned to the `pf_app` user

Figure 11 shows that `pf_app` has only the required runtime privileges. This verifies that the backend can use a limited database account instead of the MySQL root account during normal application execution.

5.8 Migrations

The `database/migrations/` folder is prepared for incremental SQL scripts used to evolve the database schema after deployment without discarding existing data. A migration file represents a discrete database change, such as adding a new column, creating an index, inserting seed data for a new feature, or modifying an existing constraint.

In local development, the database can be rebuilt from the canonical SQL files because no production data needs to be preserved. In the Railway cloud deployment, however, rebuilding from scratch may destroy live user data. Migrations are therefore the safer mechanism for applying database changes to an already-running environment.

6 Backend Implementation

6.1 Backend Overview

The backend of the system is implemented using FastAPI. It provides RESTful API endpoints that connect the frontend application with the MySQL database. The backend is responsible for request validation, authentication, authorization, business logic processing, and database access.

The backend follows a layered architecture. API routers receive requests from the frontend, the service layer handles business rules, and the repository layer communicates with the MySQL database. This structure improves maintainability because API logic, business logic, and SQL operations are separated clearly.

6.2 Backend Architecture Flow

Figure 12 shows the main request processing flow in the backend. A request from the frontend is first handled by a FastAPI router. The request is then validated and passed to the service layer. The service layer applies business rules and calls the repository layer to access the database. Finally, the result is returned to the frontend as a JSON response.

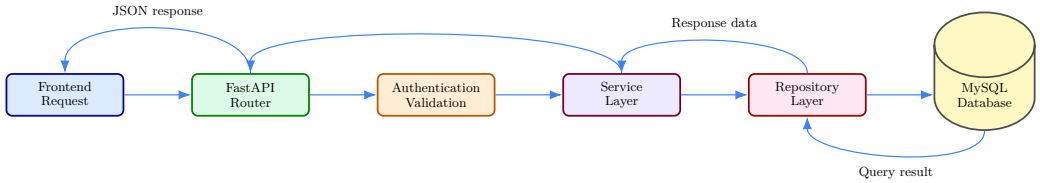

Figure 12: Backend architecture and request processing flow

6.3 Main Backend Components

The backend contains several route groups that support the main features of the system, including authentication, dashboard, transactions, imports, reports, budgets, alerts, goals, and user management. These routers expose API endpoints under the `/api/v1` prefix.

The service layer coordinates the main business logic of the system. It validates user actions, checks ownership rules, applies financial rules, and prepares response data. For example, when a user creates an expense, the backend checks whether the selected account belongs to the current user and whether the submitted amount is valid.

The repository layer separates database operations from API logic. It executes SQL queries, retrieves data from MySQL tables and views, and returns the results to the service layer.

This makes the backend easier to maintain because database access logic is not mixed directly with route handlers.

6.4 Backend Responsibilities

The backend supports the following responsibilities:

- **Authentication and authorization:** Handles login, logout, session validation, OTP-based recovery, and role-based access control.
- **Financial operations:** Handles income, expenses, budgets, saving goals, and transaction import workflows.
- **Reporting:** Retrieves dashboard and report data from SQL views and database queries.
- **Validation and error handling:** Handles invalid credentials, expired sessions, unauthorized access, invalid transaction amounts, insufficient balance, duplicate values, and missing records.

Overall, the backend acts as the main bridge between the user interface and the database. It ensures that frontend actions are validated, processed, and stored correctly in MySQL.

7 Frontend Implementation

7.1 Frontend Overview

The frontend is implemented using Next.js, TypeScript, and Tailwind CSS. It provides the user interface for the Personal Finance Management System and allows users to interact with the main features of the application, such as authentication, dashboard monitoring, transaction management, budget planning, saving goals, reports, imports, and administrator user management.

The frontend communicates with the FastAPI backend through REST API calls. Data returned from the backend is displayed through pages, forms, tables, cards, progress bars, and charts. Since the detailed user workflow is demonstrated in the project video, this section focuses on the main frontend structure and interaction flow.

7.2 Frontend Interaction Flow

Figure 13 shows how the frontend interacts with the backend and database. Users interact with Next.js pages, which use reusable UI components and API client functions. API requests are sent to the FastAPI backend, and the returned data is rendered back to the interface.

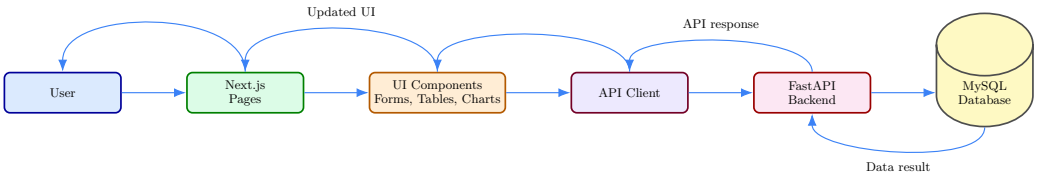

Figure 13: Frontend interaction and API communication flow

7.3 Main Frontend Features

The frontend is organized around the main pages of the application. The authentication pages allow users to log in, recover accounts, and manage access. The dashboard page provides a summary of income, expenses, balances, budgets, and saving progress. The transaction page allows users to create, update, delete, and import income or expense records.

The budget page displays monthly budget settings, planned amounts, actual spending, and warning or exceeded statuses. The saving goals page allows users to track long-term saving

or debt repayment goals. The reports page presents financial visualizations such as income versus expense, category spending, budget comparison, and balance history. The admin page supports user management for administrator accounts.

7.4 User Interface Design

The interface uses cards, forms, tables, progress indicators, and charts to make financial data easier to understand. Forms are used for data input, tables are used for transaction and user lists, and charts are used for reporting and visual analysis.

The frontend is localized in Vietnamese for labels, buttons, messages, date formatting, and currency display. This makes the system easier to use for the intended users. Administrator-only features are separated from normal user features to ensure that role-based access control is reflected in the user interface.

Overall, the frontend provides a clear and practical interface for personal finance tracking. It allows users to interact with the database-backed features through a web application, while the detailed workflow is demonstrated in the accompanying project video.

8 Testing and Evaluation

8.1 Testing Objectives

The purpose of testing is to verify that the Personal Finance Management System works correctly across the database, backend, frontend, and deployment environments. Since the project focuses on database design and implementation, database testing plays an important role in confirming that tables, constraints, views, stored procedures, functions, triggers, and database users work as expected.

The testing process also verifies that the FastAPI backend can communicate with the MySQL database, the frontend can call backend APIs successfully, and the system can run in both local and cloud environments. The detailed frontend workflow is presented in the project demonstration video, while this report focuses on selected testing evidence and system-level verification.

8.2 Database Testing

Database testing was performed using MySQL query results. The main objective was to verify that the SQL implementation correctly creates and manages the database objects required by the system.

The database testing covered the following aspects:

- **Table creation:** verifying that all required tables and views are created in the `Personal_Finance` database.
- **Table structure:** checking important tables such as `BankAccounts`, `Income`, `Expenses`, and `Users`.
- **Reporting views:** testing views such as `vw_budget_vs_actual_monthly` and `vw_account_balance_history`.
- **Stored procedures:** executing procedures such as `sp_monthly_closure_summary`.
- **User-defined functions:** testing financial calculation functions such as `fn_total_income_by_user`, `fn_total_expense_by_user`, and `fn_net_saving_by_user`.
- **Triggers:** verifying that account balances are updated automatically after transaction changes.
- **Database privileges:** checking the privileges assigned to the `pf_app` database user.

The MySQL query results for these database tests are already presented in the SQL Implementation chapter. Therefore, this chapter does not repeat all database screenshots, but summarizes the testing coverage and focuses on application-level and deployment-level verification.

8.3 Backend Testing

Backend testing verifies that the FastAPI application can process requests correctly and return valid responses. The tested backend areas include authentication, transaction management, budget operations, goal management, report generation, import workflow, and administrator user management.

The backend testing focused on the following points:

- The backend service starts successfully.
- The health check endpoint returns a valid response.
- API routes are available through FastAPI Swagger UI.
- Protected routes require authentication.
- Invalid input is rejected with appropriate error responses.
- Backend APIs can retrieve and update data from the MySQL database.

The following screenshot shows the backend health check response.

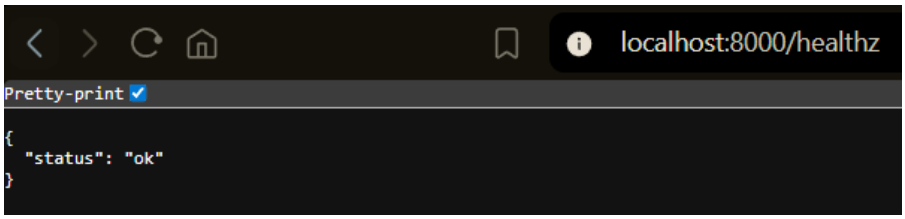

Figure 14: Backend health check response

Figure 14 confirms that the backend service is running and can respond to requests.

The following screenshot shows the FastAPI Swagger UI.

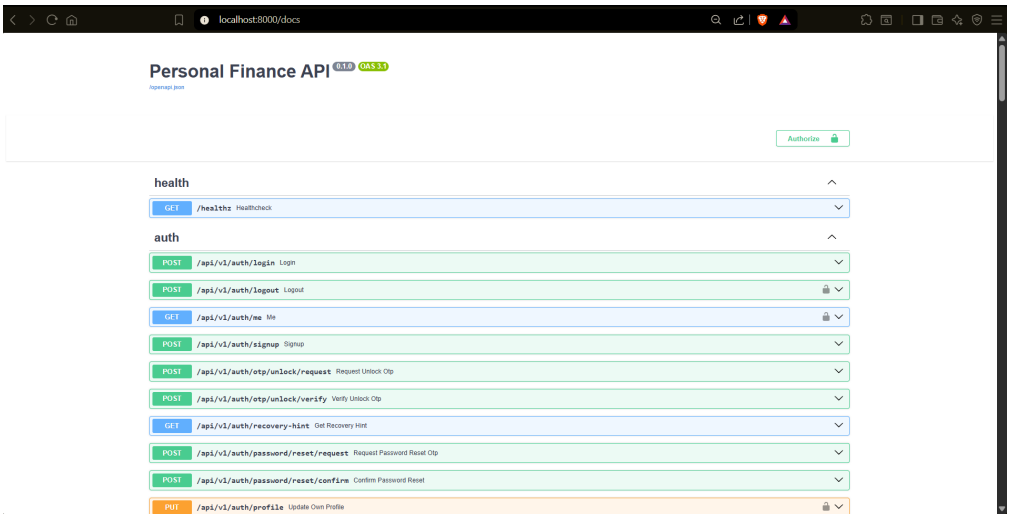

Figure 15: FastAPI Swagger UI showing available backend endpoints

Figure 15 shows the available backend API endpoints. These endpoints support authentication, dashboard data, transactions, budgets, goals, reports, imports, and user management.

8.4 Frontend Testing

Frontend testing verifies that the user interface can display data correctly and interact with backend APIs. The main tested workflows include login, dashboard display, transaction management, budget tracking, saving goal monitoring, report visualization, transaction import, and administrator user management.

Since the detailed frontend workflow is shown in the project demonstration video, this report only includes selected frontend evidence. The screenshot below shows the dashboard after a successful login.

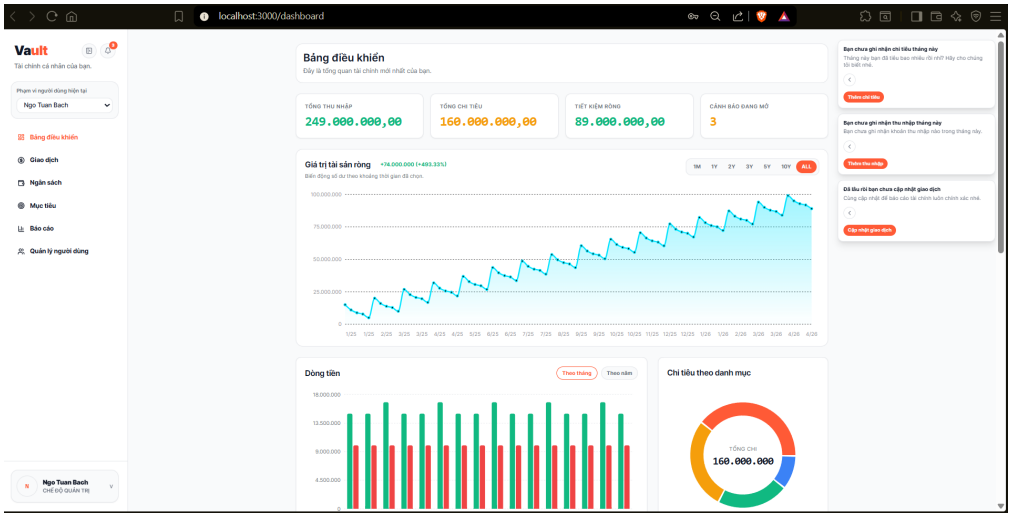

Figure 16: Dashboard page after successful login

Figure 16 confirms that the frontend can authenticate a user, retrieve financial data from the backend, and display summary information on the dashboard.

8.5 Deployment Testing

Deployment testing verifies that the system can run in both local and cloud environments. In local development, Docker Compose is used to run the MySQL database, FastAPI backend, and Next.js frontend. In the cloud environment, Railway is used to host the MySQL database, backend service, and frontend service.

The deployment testing focused on the following points:

- The MySQL database service starts successfully.
- The backend can connect to the MySQL database using environment variables.
- The frontend can call the backend API through the configured API URL.
- The deployed application can be accessed through the public Railway URL.
- Database credentials and runtime configuration are managed through environment variables.

Detailed local setup commands, Docker Compose commands, Railway variables, and troubleshooting steps are provided in the project README. This report only summarizes the deployment approach and includes one deployment verification screenshot.

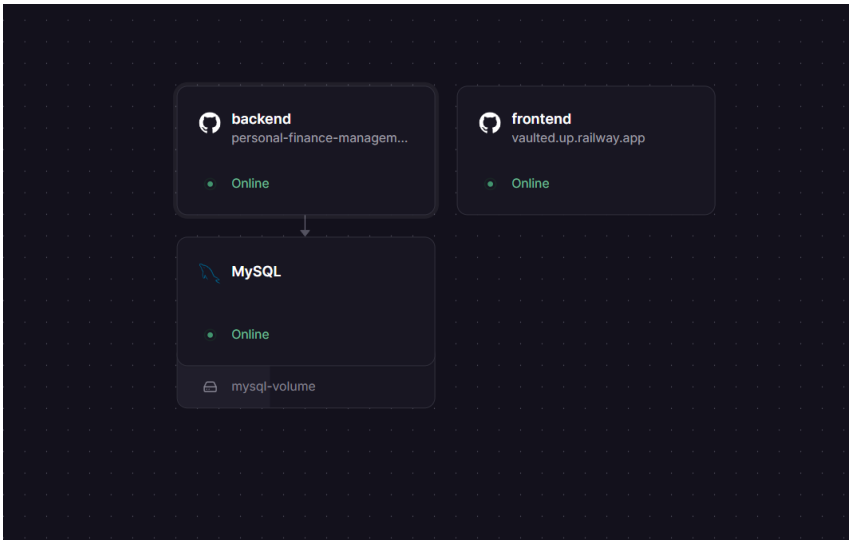

Figure 17: Railway deployment status for frontend, backend, and MySQL services

Figure 17 shows that the required services are deployed successfully on Railway.

8.6 Test Case Summary

Table 13 summarizes the main test cases used to evaluate the system. These test cases cover the most important workflows of the application.

Table 13: Sample test case summary

Test Case	Input / Action	Expected Result	Status
Admin login	Enter valid admin email and password.	The user is authenticated and redirected to the dashboard.	Pass
Invalid login	Enter an incorrect password.	The login request is rejected and an error message is shown.	Pass
Add income	Submit a valid income transaction.	The income record is saved and the selected account balance increases.	Pass
Add expense	Submit a valid expense transaction.	The expense record is saved and the selected account balance decreases.	Pass
Insufficient balance	Submit an expense amount greater than the account balance.	The transaction is rejected by the system.	Pass
Update transaction	Edit an existing income or expense transaction.	The transaction and account balance are updated correctly.	Pass
Delete transaction	Delete an existing income or expense transaction.	The transaction is removed and the account balance is adjusted.	Pass
Budget warning	Spending reaches the warning threshold.	The budget warning status is displayed.	Pass
Budget exceeded	Spending exceeds the planned budget amount.	The exceeded status is displayed.	Pass

Continued on next page

Table 13: Sample test case summary (continued)

Test Case	Input / Action	Expected Result	Status
Goal contribution	Add a contribution to an active saving goal.	The goal progress is updated correctly.	Pass
Transaction import	Upload and confirm a valid CSV or Excel file.	The transaction rows are parsed, previewed, and imported.	Pass
Report display	Open the reports page.	Financial charts and summaries are displayed correctly.	Pass
Admin user management	Update a user's role or account status.	The user information is updated correctly.	Pass
Railway deployment	Open the deployed application URL.	The frontend, backend, and MySQL services work together successfully.	Pass

8.7 Evaluation

The testing results show that the system satisfies the main functional requirements. Users can log in, manage income and expense transactions, track account balances, configure budgets, monitor saving goals, import transaction files, and view financial reports. Administrator-level user management is also supported.

At the database level, MySQL testing confirms that tables, constraints, views, stored procedures, user-defined functions, triggers, and database privileges work correctly. The trigger tests confirm that account balances are synchronized with transaction changes, while the view and function tests confirm that financial summaries can be calculated at the database layer.

At the application level, backend and frontend testing confirm that the FastAPI backend can communicate with MySQL and that the Next.js frontend can retrieve and display financial data through API calls. Deployment testing confirms that the system can run locally through Docker Compose and globally through Railway.

Overall, the system is functional, testable, and suitable for the intended purpose of personal finance management.

9 Discussion

9.1 Security and Data Integrity Summary

Security and data integrity are important aspects of the system because the application stores personal financial information. At the application level, user authentication is handled through password hashing, session tokens, OTP-based account recovery, and role-based access control. Passwords, OTP codes, and session tokens are not stored in plain text.

At the database level, the system uses MySQL constraints and security configuration to protect data consistency. Primary keys uniquely identify records, foreign keys enforce relationships between tables, unique constraints prevent duplicate important values, and check constraints validate financial amounts and period values. Triggers are used to keep account balances synchronized when income or expense records change.

The system also applies the principle of least privilege at the database level. The `pf_app` database user is intended for backend runtime operations and receives only the required permissions, such as `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `SHOW VIEW`, and `EXECUTE`. This avoids using the MySQL root account for normal application execution.

9.2 Achievements

The project successfully implements a database-backed personal finance management system. The main achievements include:

- A normalized MySQL database schema for users, accounts, transactions, budgets, goals, authentication, imports, and reporting.
- Core ERD and supporting ERDs for authentication/security and transaction import/linking.
- SQL implementation with tables, constraints, indexes, views, stored procedures, user-defined functions, triggers, and security users.
- A FastAPI backend that exposes RESTful API endpoints and connects the frontend to the database.
- A Next.js frontend that provides user-facing pages for dashboard, transactions, budgets, goals, reports, imports, and admin management.
- Local deployment using Docker Compose and cloud deployment using Railway.
- Testing evidence through MySQL query results, backend API verification, frontend workflow testing, and deployment verification.

These achievements show that the project meets the main requirements of a database management system project and demonstrates practical use of relational database design and SQL implementation.

9.3 Limitations

Although the system is functional, it still has several limitations. First, the system does not connect directly to real banking APIs. Bank accounts in the system are internal records created by users, and transactions are entered manually or imported from files. Second, the transaction import feature depends on the format and quality of the uploaded CSV or Excel file. If the file structure is inconsistent, additional preprocessing may be required.

The current system also focuses mainly on rule-based reporting and visualization. It does not include advanced predictive analytics, automated spending recommendations, or AI-based financial planning. In addition, the system is implemented as a web application and does not yet provide a dedicated mobile application.

9.4 Future Improvements

Future improvements can extend the system in several directions:

- Integrate with real banking APIs to automatically import transactions.
- Add recurring transaction support for rent, subscriptions, salaries, and fixed expenses.
- Improve transaction import by supporting more file formats and smarter column mapping.
- Add notifications for budget warnings, goal deadlines, and unusual spending patterns.
- Implement AI-based spending prediction and cash flow forecasting.
- Add export features for PDF and Excel financial reports.
- Develop a mobile application for easier personal finance tracking.
- Improve admin monitoring and audit log visualization.

These improvements would make the system more automated, intelligent, and suitable for long-term personal finance management.

10 Conclusion

This project developed a Personal Finance Management System using MySQL, FastAPI, and Next.js. The main objective of the system is to help users manage their personal financial data, including bank accounts, income, expenses, budgets, saving goals, transaction imports, reports, and authentication-related information.

From the database perspective, the project successfully designed and implemented a relational database named `Personal_Finance`. The database design includes a Core ERD for the main financial entities, an Authentication and Security Supporting ERD, and a Transaction Import and Linking Supporting ERD. These ERDs show how the system organizes financial records, authentication data, import workflows, and goal-related transactions.

The SQL implementation demonstrates the use of important database management concepts. The schema defines tables, primary keys, foreign keys, unique constraints, check constraints, and indexes. Views are used to simplify reporting queries, stored procedures are used to centralize common database operations, user-defined functions are used for reusable financial calculations, and triggers are used to keep account balances synchronized with income and expense transactions. The `security.sql` file also introduces database-level users such as `pf_app` and `pf_readonly`, supporting the principle of least privilege.

The backend implementation uses FastAPI to provide RESTful APIs between the frontend and the MySQL database. It handles authentication, validation, business logic, financial operations, reporting, import processing, and admin user management. The frontend implementation uses Next.js, TypeScript, and Tailwind CSS to provide a web interface for users to interact with the system. The web interface supports dashboard monitoring, transaction management, budget planning, saving goal tracking, reports, import preview, and admin management.

Testing confirmed that the system works correctly across the database, backend, frontend, and deployment environments. MySQL query results verified the creation of tables, views, stored procedures, functions, triggers, and database privileges. Backend testing confirmed that API endpoints are available and functional. Frontend testing confirmed that users can complete the main workflows through the web interface. Deployment testing confirmed that the system can run locally using Docker Compose and globally using Railway.

Overall, the project achieved its main goals and demonstrates a complete database-backed application. It applies relational database design principles, implements advanced SQL objects, connects the database with a backend API, and provides a practical frontend for personal finance management. Although the system does not yet integrate with real banking APIs or advanced prediction features, it provides a solid foundation for future improvements such as automated bank synchronization, recurring transactions, spending prediction, notifications, and mobile support.

References

[1] Oracle Corporation. *MySQL 8.4 Reference Manual*. Available at: https://dev.mysql.com/doc/

[2] FastAPI. *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/

[3] Vercel. *Next.js Documentation*. Available at: https://nextjs.org/docs

[4] Meta Open Source. *React Documentation*. Available at: https://react.dev/

[5] Tailwind Labs. *Tailwind CSS Documentation*. Available at: https://tailwindcss.com/docs

[6] Docker Inc. *Docker Documentation*. Available at: https://docs.docker.com/

[7] Railway. *Railway Documentation*. Available at: https://docs.railway.com/

[8] Oracle Corporation. *MySQL Connector/Python Developer Guide*. Available at: https://dev.mysql.com/doc/connector-python/en/

[9] Recharts. *Recharts Documentation*. Available at: https://recharts.org/

[10] OWASP Foundation. *OWASP Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[11] Ngo Tuan Bach. *Personal Finance Management System Repository*. Available at: https://github.com/tuanbachngo/Personal-Finance-management

Appendix

A Source Code Structure

Listing 2: Project source code structure

```
Personal-Finance-management/
  database/
    schema.sql
    sample_data.sql
    views.sql
    procedures.sql
    functions.sql
    triggers.sql
    security.sql
    migrations/

  backend/
    app/
      main.py
      config.py
      deps.py
      routers/

  frontend/
    src/
      app/
      components/
      lib/
      providers/
      types/

  python_app/
    services/
    repositories/
    db_connection.py

  ops/
    bootstrap_auth.py
    docker/
      reset_database.sh
      apply_views.sh

  docs/
    assignment files
    report files
    ERD images
```

B SQL Script Inventory

Table 14: SQL script inventory

SQL File	Purpose
`schema.sql`	Creates the `Personal_Finance` database, tables, primary keys, foreign keys, unique constraints, check constraints, and indexes.
`sample_data.sql`	Inserts representative sample data for users, banks, bank accounts, transactions, budgets, saving goals, goal contributions, and category rules.

Continued on next page

Table 14: SQL script inventory (continued)

SQL File	Purpose
`views.sql`	Creates reporting and dashboard views such as budget comparison, account balance history, category-wise spending, and saving goal progress.
`procedures.sql`	Creates stored procedures for income, expense, budget, and monthly financial summary operations.
`functions.sql`	Creates user-defined functions for reusable financial calculations such as total income, total expense, net saving, and account balance calculation.
`triggers.sql`	Creates triggers that automatically update account balances and prevent invalid expense transactions when the selected account has insufficient balance.
`security.sql`	Defines database-level MySQL users and privileges, including limited runtime access for `pf_app` and read-only access for `pf_readonly`.
`migrations/`	Contains or prepares incremental SQL scripts for updating an existing database without resetting all data.

C Full ERD

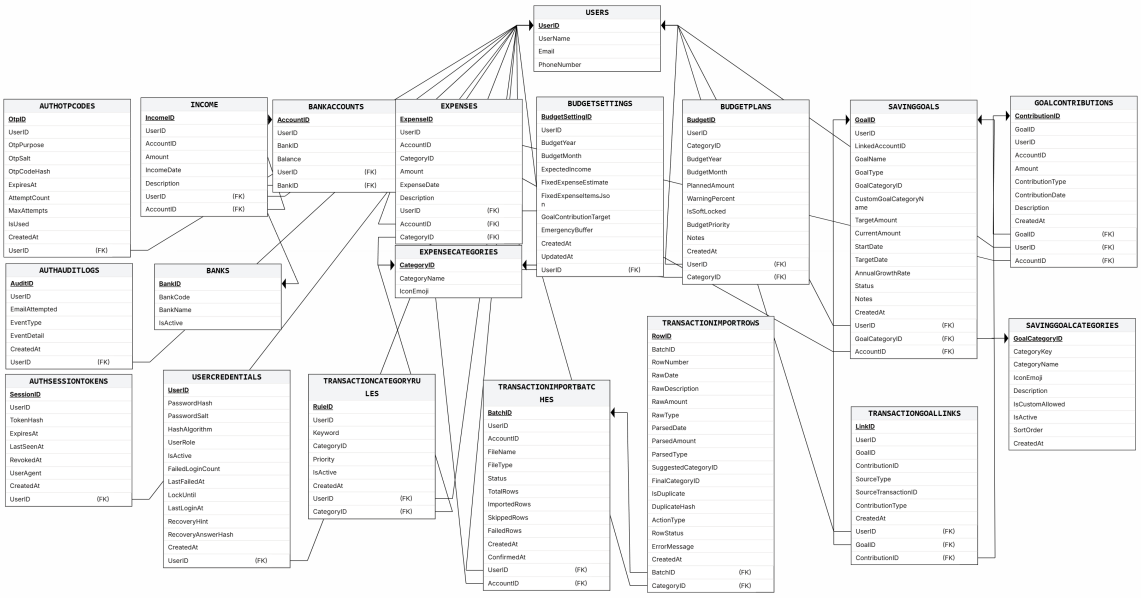

Figure 18: Full ERD of the Personal_Finance database

D Repository and Demo Links

- GitHub Repository: https://github.com/tuanbachngo/Personal-Finance-management
- Demo Video: https://youtu.be/eOIub3bRHTI
- Deployed Website: https://vaulted.up.railway.app/login